APOSTILA DO ALUNO — AULA 14

Sprint 1: Descoberta

Curso: Técnico em Desenvolvimento de Sistemas | Turma: 103513 | Disciplina: Modelagem de Sistemas (82h + 04h planejamento)

Data: 30/10/2026 (sexta-feira) | Horário: 07:30 às 11:30

Objetivos desta sprint

- Participar da Sprint Planning e pegar uma tarefa pelo portal, com o comando /eu-pego.
- Entender o Business Model Canvas e os seus 9 blocos, e montar o Canvas do TechBurger.
- Preparar e conduzir uma entrevista com a cliente, usando as técnicas da Aula 2.
- Transformar a fala da cliente em regras de negócio, requisitos funcionais e não funcionais, com ID e origem.
- Entregar a sua parte por pull request e participar da Sprint Review e da retrospectiva.

Como usar esta apostila

Esta é a primeira sprint de verdade do projeto. A apostila segue a ordem do dia: a planning (seção 1), o Canvas (seções 2 e 3), a entrevista (seção 4), a passagem da fala para o requisito (seção 5), a entrega (seções 6 e 7) e a review (seção 8). Se você faltou ou não terminou a tarefa na aula, as seções 4 a 7 bastam para fazer a sua parte em casa.

1. Sprint Planning: o objetivo e a sua tarefa

O objetivo da Sprint 1 é entender o negócio do TechBurger antes de construir qualquer tela. Para isso, a turma monta junta o Canvas do negócio, entrevista a cliente (a gerente Fernanda, representada pelo professor) e cada dev registra UMA parte do seu módulo. Cada squad tem quatro arquivos na pasta docs/requisitos/<squad>/ do repositório, e cada arquivo é a tarefa de um dev.

Squad · Issues	entrevista.md	regras.md	rf.md	rnf.md
Cardápio (#2–5)	1 dev: Entrevista	1 dev: Regras	1 dev: RF	1 dev: RNF
Carrinho (#6–9)	1 dev: Entrevista	1 dev: Regras	1 dev: RF	1 dev: RNF
Pagamento (#10–13)	1 dev: Entrevista	1 dev: Regras	1 dev: RF	1 dev: RNF
Cozinha (#14–17)	1 dev: Entrevista	1 dev: Regras	1 dev: RF	1 dev: RNF
Cliente (#18–20)	1 dev: entrevista + regras		1 dev: RF	1 dev: RNF

Cada arquivo fica em docs/requisitos/<squad>/ · 1 dev = 1 tarefa = 1 pull request · 4 + 4 + 4 + 4 + 3 = 19

Figura 1 — As 19 tarefas da Sprint 1: uma por dev.

Arquivo	O que registrar	Formato

entrevista.md	As perguntas do squad e as respostas da Fernanda, com os números que ela disser	Tabela pergunta / resposta + resumo + dúvidas
regras.md	As regras de negócio do módulo	Condição / ação / exceção / origem (Aula 1)
rf.md	Os requisitos funcionais do módulo	"O sistema deve..." + origem + MoSCoW (Aula 2)
rnf.md	Os requisitos não funcionais do módulo	Categoria + qualidade com número + como medir (Aula 2)

O Squad Cliente tem 3 devs: nele, uma tarefa junta a entrevista e as regras. Todos participam da entrevista e anotam, mas só quem pegou a tarefa de entrevista registra no entrevista.md.

1.1 Como pegar a sua tarefa

As tarefas ficam na página da Sprint do portal (aprendendodev.com.br/tarefas.html). Ela mostra o objetivo da sprint, um placar (Livre, Fazendo, Em revisão, Pronto) e as tarefas de cada squad. Digite o seu usuário do GitHub em "Minha tarefa": o portal mostra o seu squad e o seu próximo passo (card, pegar, fazer e enviar, revisão, pronto), sempre com o botão certo. A página se atualiza sozinha a cada minuto.

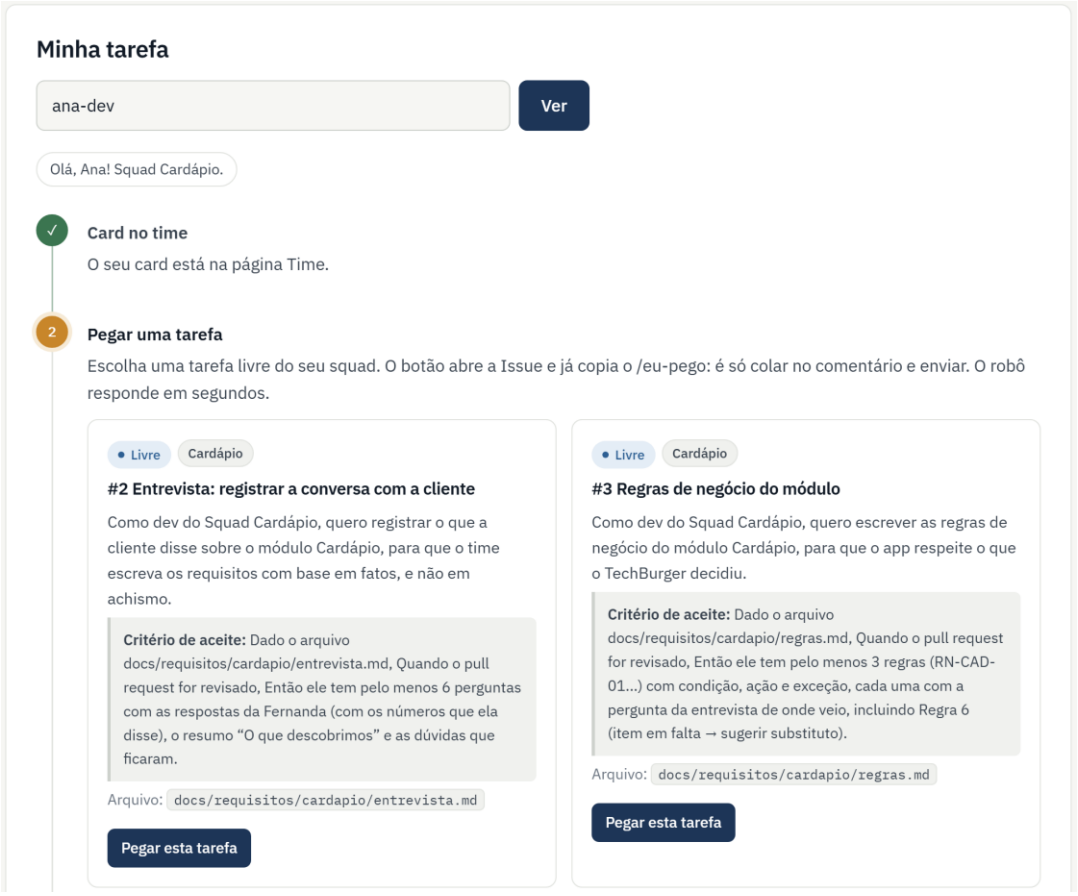


Figura 2 — Minha tarefa: as tarefas livres do seu squad, cada uma com o botão Pegar esta tarefa.

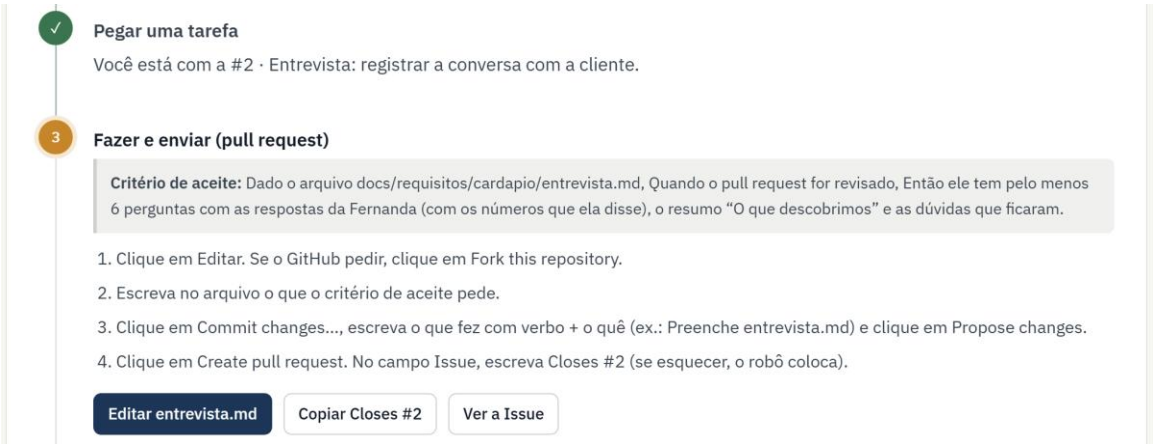


Figura 3 — Depois de pegar: o critério de aceite e o botão Editar, que abre o arquivo da tarefa.

1. Combine com o seu squad quem faz o quê (2 minutos).

2. Em Minha tarefa, clique em Pegar esta tarefa numa tarefa Livre do SEU squad. A Issue abre no GitHub e o /eu-pego já fica copiado.

3. No campo de comentário da Issue, cole (Ctrl+V) o /eu-pego e clique em Comment.

4. Em alguns segundos, o robô do time responde com o link para editar o arquivo. A tarefa fica com você, ganha a etiqueta fazendo e aparece como Fazendo no portal.

5. Mudou de ideia? Comente /largar e a tarefa volta a ficar Livre.

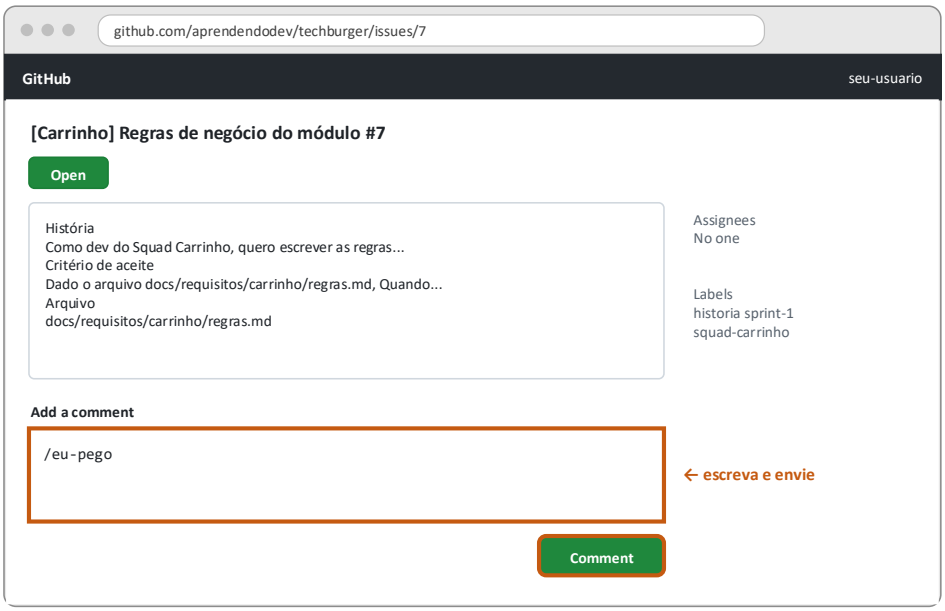


Figura 4 — Comentar /eu-pego na Issue (esquema da tela).

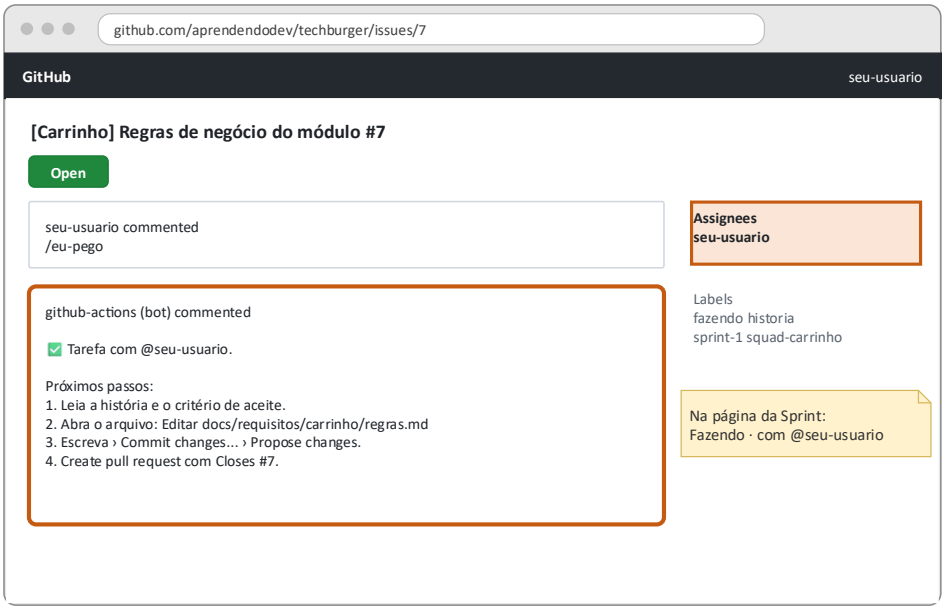


Figura 5 — A resposta do robô: a tarefa fica com você (esquema da tela).

O robô recusa o pedido em três casos, sempre explicando o motivo: a tarefa já tem dono; a tarefa é de outro squad (ele confere o squad no seu card da pasta equipe/); ou você já tem outra tarefa aberta nesta sprint — a regra do quadro é uma tarefa por vez.

2. Business Model Canvas

Software existe para resolver um problema de um negócio. Quem entende o negócio faz as perguntas certas e não constrói coisa que ninguém precisa. Por exemplo: sem saber que o TechBurger perde dinheiro preparando lanche de pedido que nunca foi pago, ninguém pensaria na regra “a cozinha só recebe pedido pago”.

O Business Model Canvas, criado por Alexander Osterwalder, descreve um negócio inteiro numa página, em 9 blocos. A grade tem uma lógica: do lado DIREITO fica o cliente (quem é, como chegamos até ele, como nos relacionamos); no CENTRO, a proposta de valor; do lado ESQUERDO, a operação (recursos, atividades, parceiros); EMBAIXO, o dinheiro (custos à esquerda, receitas à direita).

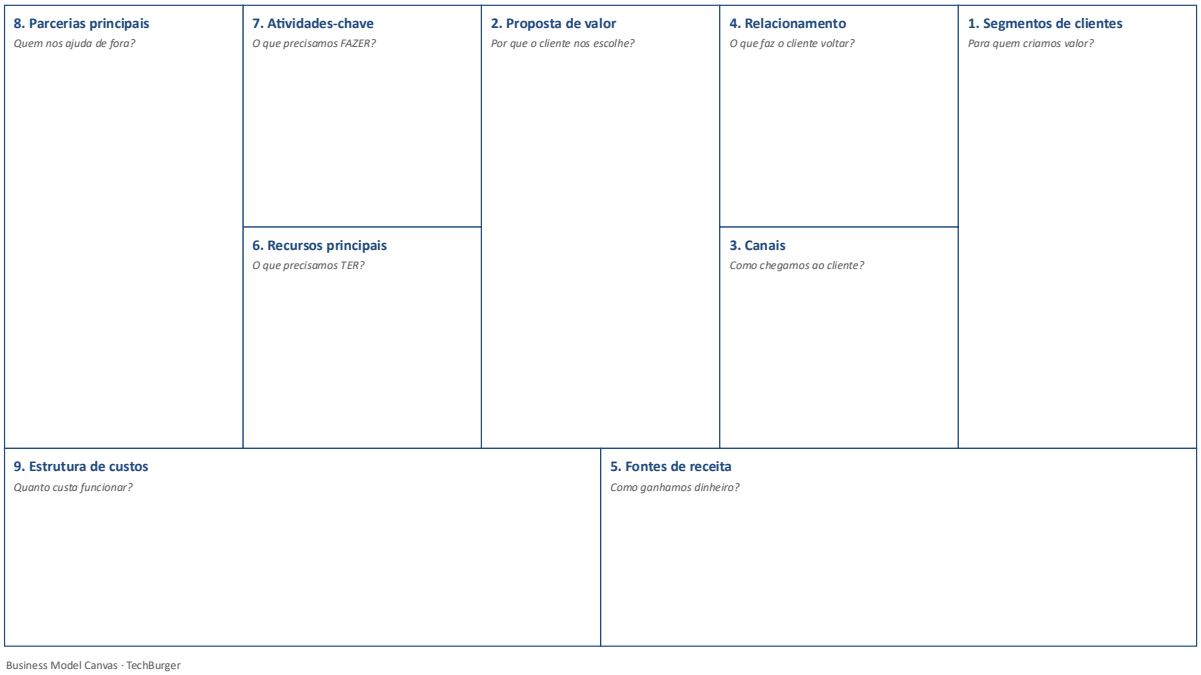


Figura 6 — A grade do Canvas: o número de cada bloco é a ordem de preenchimento.

#	Bloco	Pergunta que responde	Ligação com o app
1	Segmentos de clientes	Para quem criamos valor?	Daqui nasce a persona (Aula 10)
2	Proposta de valor	Por que o cliente nos escolhe?	As funcionalidades principais vêm daqui
3	Canais	Como chegamos ao cliente?	O app é um canal novo
4	Relacionamento	O que faz o cliente voltar?	Fidelidade e status do pedido
5	Fontes de receita	Como ganhamos dinheiro?	Pedido mínimo e desconto PIX (Regras 1 e 4)
6	Recursos principais	O que precisamos TER?	O app vira um recurso
7	Atividades-chave	O que precisamos FAZER bem?	Viram processos (diagrama de atividades)
8	Parcerias principais	Quem nos ajuda de fora?	Sistemas externos da Aula 11
9	Estrutura de custos	Quanto custa funcionar?	Ex.: taxa do gateway por pedido

2.1 Os 9 blocos, um a um

1. Segmentos de clientes — São os grupos de pessoas que o negócio atende, cada um com necessidades parecidas. “Todo mundo” não é segmento: quanto mais específico, melhores as decisões. É deste bloco que sai a persona da Aula 10 — e, no app, as telas que cada grupo vai usar.

2. Proposta de valor — É o motivo pelo qual o cliente escolhe o negócio e não o concorrente: o problema que ele resolve ou o ganho que ele entrega. Tem que ser concreta e verificável. As funcionalidades mais importantes do app nascem aqui.

3. Canais — São os caminhos pelos quais a proposta chega ao cliente: divulgação, venda e entrega. Um app de pedidos é um canal; o balcão e a entrega própria também são.

4. Relacionamento com clientes — É o tipo de vínculo que o negócio mantém com cada segmento: atendimento pessoal, autoatendimento, comunidade, fidelidade. A pergunta-chave é: o que faz o cliente voltar?

5. Fontes de receita — É como o negócio ganha dinheiro: o que o cliente paga, quanto e de que forma. Preços, taxas, formas de pagamento e descontos entram aqui — e quase sempre viram regras de negócio.

6. Recursos principais — É o que o negócio precisa TER para funcionar: recursos físicos (cozinha), humanos (equipe), intelectuais (as receitas da casa) e tecnológicos (o app). Teste: se faltar isso, o negócio para?

7. Atividades-chave — É o que o negócio precisa FAZER bem, todos os dias. Escreva com verbos: preparar, entregar, atualizar, confirmar. Cada atividade vira, mais tarde, um processo no diagrama de atividades.

8. Parcerias principais — São as outras empresas que fazem parte do trabalho: fornecedores e prestadores de serviço. No mundo do software, muitas parcerias são sistemas externos, como o gateway de pagamento e o serviço de mapas da Aula 11.

9. Estrutura de custos — É quanto custa fazer tudo funcionar. Os custos saem dos recursos, das atividades e das parcerias, por isso este é o último bloco. Uma taxa por pedido do gateway, por exemplo, ajuda a explicar por que o PIX tem desconto.

Três cuidados ao preencher. Primeiro, a proposta de valor precisa ser concreta: “qualidade” e “bom atendimento” todo mundo diz; “pedido em até 4 toques” e “5% de desconto no PIX” dizem algo. Segundo, canal não é relacionamento: o canal é o caminho até o cliente (o app, o balcão), o relacionamento é o vínculo que o faz voltar (a fidelidade). Terceiro, os custos são o último bloco porque saem dos recursos, das atividades e das parcerias.

3. Exemplo guiado: o Canvas do TechBurger

Em aula, o professor preenche o Canvas no draw.io (arquivo “Aula 14 - Canvas TechBurger (ponto de partida).drawio”) e a turma dita as respostas, bloco a bloco, na ordem dos números. Use o que já sabemos do TechBurger desde a Aula 1: as 8 regras, a persona, as integrações e os dados.

Passo 1 — segmentos de clientes

- Para quem o TechBurger cria valor?
- Lembrem da entrega até 5 km (Regra 5) e da persona da Aula 10
- Clientes fiéis também são um grupo: existem por causa da fidelidade

Passo 2 — proposta de valor

- Por que o cliente escolhe o TechBurger?
- Nada de “qualidade”: o que é concreto? 4 toques, desconto PIX, combo grátis
- Acompanhar o pedido em tempo real também é valor (heurística 1, Aula 10)

Passo 3 — canais

- Como o TechBurger chega até o cliente?
- O app que vamos construir é um CANAL novo
- A entrega própria também é canal: a proposta chega quente na casa do cliente

Passo 4 — relacionamento

- O que faz o cliente voltar?
- Fidelidade (Regra 2), status do pedido, atendimento no balcão
- Avaliar o pedido mostra que a opinião do cliente importa

Passo 5 — fontes de receita

- Como o TechBurger ganha dinheiro?
- Venda dos itens + taxa de entrega
- Formas de pagamento e o pedido mínimo (Regra 1) também entram aqui

Passo 6 — recursos principais

- O que o TechBurger precisa TER para funcionar?
- Cozinha, receitas, equipe e entregadores
- O app e o painel da cozinha passam a ser recursos

Passo 7 — atividades-chave

- O que o TechBurger precisa FAZER bem todo dia?
- Verbos: preparar, entregar, atualizar o cardápio, confirmar pagamentos
- Cada atividade vira um processo: diagrama de atividades (Aula 7)

Passo 8 — parcerias principais

- Quem ajuda o TechBurger de fora?
- Fornecedores de ingredientes e embalagens
- Os sistemas externos da Aula 11: gateway de pagamento e serviço de mapas

Passo 9 — estrutura de custos

- Quanto custa fazer tudo isso funcionar?
- Olhem para a esquerda do Canvas: cada recurso, atividade e parceria custa algo
- A taxa do gateway por pedido é custo: por isso o PIX tem desconto

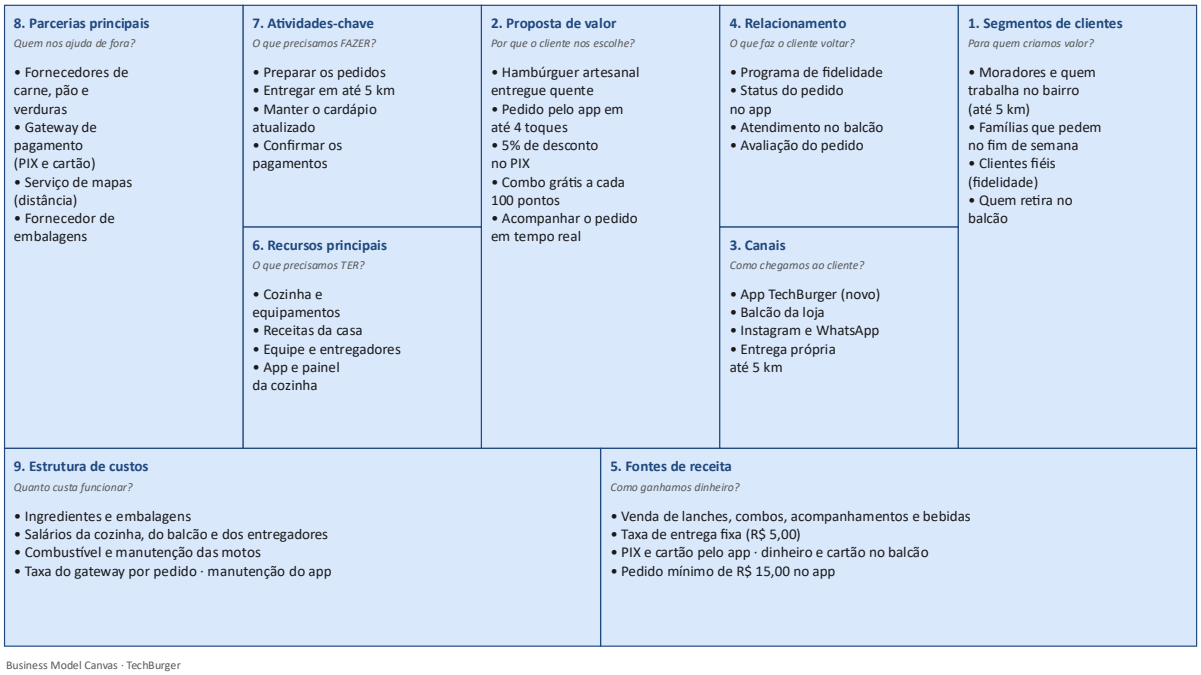


Figura 7 — O Canvas do TechBurger completo.

O Canvas diz POR QUE o app existe; a entrevista vai dizer O QUE ele precisa fazer. Cada bloco gera perguntas: se o Canvas tem “taxa de entrega”, alguém precisa perguntar quanto custa, quem paga e até onde vale. O arquivo final vai para a pasta docs/canvas/ do repositório.

Bloco do Canvas	Squads que ele alimenta
Proposta de valor (4 toques, desconto PIX, combo grátis)	Cardápio, Carrinho, Pagamento e Cliente
Relacionamento (fidelidade, status em tempo real)	Cliente e Cozinha
Fontes de receita (PIX, cartão, taxa de entrega, mínimo)	Pagamento, Carrinho e Cliente
Parcerias (gateway de pagamento, mapas)	Integrações (Aula 11): Pagamento e Cliente
Atividades-chave (preparar, entregar)	Cozinha e os fluxos da Sprint 4

4. A entrevista com a cliente

A entrevista é a técnica de levantamento certa quando há poucas pessoas e o assunto é complexo (Aula 2). Ela funciona bem quando segue algumas regras:

- **Comece aberto, depois feche.** “Como o pedido chega na cozinha hoje?” abre a conversa; “Quanto tempo leva o preparo?” fecha num número.
- **Pergunte pela exceção.** “E quando não dá?”, “Tem caso em que isso não vale?”. A exceção é a terceira parte de toda regra, e quase nunca aparece sozinha.
- **Peça números.** Tempo, quantidade, valor, distância: eles viram requisitos não funcionais.
- **Anote com as palavras da cliente.** Não interprete na hora; a interpretação vem depois, no regras.md e no rf.md.
- **Não pergunte sobre tecnologia.** A cliente sabe do negócio. “Você quer um banco de dados?” não ensina nada.

Pergunta boa	Pergunta ruim
“Como o pedido chega na cozinha hoje?”	“O app tem que ser bom, né?”
“Qual o valor mínimo? E no balcão?”	“Vai ter carrinho?”
“Quanto tempo o cliente espera?”	“Pode ser rápido?”
“E se faltar um ingrediente?”	“Você quer um banco de dados?”

4.1 Durante a entrevista: como anotar

Todos do squad anotam, mesmo quem não pegou a tarefa de entrevista: uma resposta perdida faz falta para quem escreve as regras e os requisitos. Anote cada pergunta com um número, a resposta com as palavras da Fernanda e, principalmente, os números que ela disser (valores, tempos, quantidades, distâncias). Se ela disser algo vago, como “rápido” ou “muitos”, peça na hora: “quanto é rápido?”, “quantos são muitos?”. Se ela mencionar uma exceção, marque com um asterisco: ela vai virar a terceira parte de uma regra.

4.2 Depois da entrevista: organizar

Antes de escrever, o squad compara as anotações em 2 minutos. Quem faz o entrevista.md registra as 6 perguntas e respostas, o resumo “O que descobrimos” e as dúvidas que ficaram. Quem faz o regras.md procura os “se... então”. Quem faz o rf.md transforma cada coisa que o sistema precisa fazer numa linha “O sistema deve...”. Quem faz o rnf.md procura os números. As dúvidas que ficaram vão para a próxima conversa com a cliente — elas não são erro, são parte do trabalho de descoberta.

4.3 Perguntas de partida

Cada squad começa com estas 4 perguntas e completa com mais 2 suas, chegando a 6. Ordem da entrevista: Cardápio, Carrinho, Pagamento, Cozinha e Cliente, com 8 minutos por squad. A turma toda ouve: uma resposta pode servir a outro squad.

Squad	Perguntas de partida
Cardápio	Quais são as categorias do cardápio? · O que cada produto precisa mostrar? · Quem muda o preço? Com que frequência? · E quando acaba um ingrediente?
Carrinho	Tem valor mínimo para pedir? E no balcão? · Pode mudar a quantidade e pôr observação? · O que fazer com um pedido muito grande? · O carrinho fica guardado se fechar o app?
Pagamento	Quais formas de pagamento no app? · O desconto do PIX vale sobre o quê? · Quanto tempo o cliente tem para pagar? · Quando a cozinha pode começar?
Cozinha	Como o pedido chega na cozinha hoje? · Quais são as etapas de um pedido? · Quanto tempo leva o preparo? · O que é um pedido atrasado?
Cliente	Que dados do cliente a loja precisa? · Como funciona a fidelidade? Os pontos vencem? · Até onde entrega? Quanto custa? · O que acontece com quem cancela muito?

5. Da fala da cliente ao requisito

Depois da entrevista, cada resposta é registrada em três formatos que já conhecemos. O caminho é sempre o mesmo: pergunta, resposta, regra, requisito funcional e requisito não funcional.



Figura 8 — Da resposta da cliente ao requisito, com o exemplo do pedido mínimo.

5.1 Regra de negócio (regras.md)

Ouça o "se... então" na fala da cliente e escreva com condição, ação, exceção e origem. Cada regra tem um ID que nunca muda: RN + o código do squad + um número.

Campo	Exemplo (Squad Carrinho)
ID e nome	RN-CAR-01 — Pedido mínimo
Condição	O total dos itens está abaixo de R$ 15,00, num pedido pelo app
Ação	Impedir a finalização e mostrar quanto falta
Exceção	No balcão não há valor mínimo; a taxa de entrega não conta para o mínimo
Origem	Entrevista, pergunta 1

5.2 Requisito funcional (rf.md)

Um requisito por linha, sempre começando com "O sistema deve..." e com uma coisa só. Toda linha diz a origem (uma regra ou uma pergunta) e a prioridade MoSCoW (Must, Should, Could, Won't). Toda regra vira pelo menos um RF.

ID	Requisito funcional	Origem	Prioridade
RF-CAR-03	O sistema deve impedir a finalização de pedidos com itens abaixo de R$ 15,00	RN-CAR-01	Must
RF-CAR-04	O sistema deve mostrar quanto falta para o valor mínimo	RN-CAR-01	Must

5.3 Requisito não funcional (rnf.md)

Uma qualidade com um número que dá para medir, e o jeito de medir. "Rápido", "bonito" e "fácil" sem número não são RNF: ninguém consegue testar.

ID	Categoria	Requisito (com número)	Como medir
RNF-CAR-01	Desempenho	O total atualiza em até 1 segundo depois de mudar um item	Cronômetro, no celular
RNF-CAR-02	Usabilidade	O aviso do mínimo aparece sem precisar rolar a tela	Teste com 5 itens no carrinho

Squad	Código	Exemplos de ID
Cardápio	CAD	RN-CAD-01 · RF-CAD-01 · RNF-CAD-01
Carrinho	CAR	RN-CAR-01 · RF-CAR-01 · RNF-CAR-01
Pagamento	PAG	RN-PAG-01 · RF-PAG-01 · RNF-PAG-01
Cozinha	COZ	RN-COZ-01 · RF-COZ-01 · RNF-COZ-01
Cliente	CLI	RN-CLI-01 · RF-CLI-01 · RNF-CLI-01

5.4 Por que ID e origem importam: o rastro até o código

Os IDs nunca mudam depois do merge, porque as próximas sprints vão apontar para eles. Na Sprint 2, cada história de usuário vai dizer de qual RF nasceu; na Sprint 3, as classes vão ter os atributos que os RF pedem; nas Sprints 4 e 5, cada pull request de código vai dizer qual regra implementa; e na Sprint 7, cada RNF vira um teste. É a rastreabilidade da Aula 9, agora do começo ao fim do projeto. A origem faz o caminho de volta: se alguém perguntar "por que o app bloqueia pedido de R$ 12,00?", a resposta está no RF-CAR-03, que veio da RN-CAR-01, que veio da pergunta 1 da entrevista.

Sprint	O que vai apontar para os IDs de hoje
2 · Análise	Histórias de usuário e casos de uso ("origem: RF-CAR-03")
3 · Projeto	Atributos e métodos das classes, campos do JSON, telas do wireframe
4 e 5 · Construção	Pull requests de código ("implementa RN-CAR-01")
7 · Qualidade	Testes: cada critério de aceite e cada RNF vira um teste

5.5 Como saber se o seu arquivo ficou bom

Arquivo	Está bom quando...
entrevista.md	tem pelo menos 6 perguntas numeradas, as respostas com as palavras da cliente e os números, o resumo e as dúvidas
regras.md	cada regra tem ID, nome, condição, ação, exceção (ou "nenhuma") e origem; uma decisão por regra
rf.md	cada linha começa com "O sistema deve", trata de uma coisa só, é do seu módulo e tem origem e prioridade
rnf.md	cada linha tem categoria, um número e o jeito de medir; as categorias variam (desempenho, usabilidade, segurança...)

Um teste rápido para qualquer item: se dois colegas lessem e testassem, chegariam ao mesmo resultado? Se não, falta clareza — quase sempre falta um número ou uma exceção.

6. Exercício: encontre os problemas

Antes de escrever o seu arquivo, revise o do estagiário. Em duplas, achem os 6 problemas no rf.md que ele mandou para o Squad Carrinho. Para cada um, digam o que está errado, qual formato da Aula 1 ou 2 ele quebra, e escrevam a correção. Tempo: 5 minutos.

docs/requisitos/carrinho/rf.md · pull request do estagiário

Requisitos funcionais — Squad Carrinho

ID	Requisito funcional	Origem	Prioridade
RF-CAR-01	O sistema deve ser rápido	—	Must
RF-CAR-02	O sistema deve permitir adicionar itens e pagar com PIX	Regra 1	Must
RF-CAR-03	Carrinho com o total	Pergunta 2	Should
RF-CAR-04	O sistema deve impedir finalizar pedido abaixo de R$ 15,00		Must
RF-CAR-05	O sistema deve avisar o gerente quando o pedido tiver muitos itens	RN-CAR-02	Must
RF-CAR-05	O sistema deve guardar o carrinho por 24 horas	Pergunta 5	Could

Figura 9 — O rf.md do estagiário.

7. Mão na massa: a sua entrega

Começa com uma daily rápida (5 minutos, por squad): cada dev diz em uma frase qual é a sua tarefa e se tem algum bloqueio. O Tech Lead confere se todo mundo tem tarefa. Depois, 35 minutos para escrever e abrir o pull request.

1. Em Minha tarefa (página da Sprint), clique em Editar: o arquivo da tarefa (docs/requisitos/<squad>/...) abre no GitHub. O mesmo link está na resposta do robô na Issue. O fork você já criou na Sprint 0 (o mesmo serve para todas as tarefas). Se o GitHub pedir um fork e o botão Fork this repository não fizer nada, crie pela página Começar (botão 1. Criar o meu fork) ou em github.com/aprendendodev/techburger/fork; se nem assim funcionar, confirme o seu e-mail no GitHub (Settings › Emails).

2. Preencha seguindo o modelo que já está no próprio arquivo. Apague os comentários <!-- --> se quiser.

3. Clique em Commit changes..., escreva uma mensagem com verbo + o quê (“Registra regras do carrinho”), e clique em Propose changes (o GitHub cria uma branch nova no seu fork).

4. Clique em Create pull request: no título, o que você fez; na descrição, Closes #número-da-sua-issue e o modelo do time preenchido. Se esquecer o Closes, o robô do time acrescenta e comenta no PR; ele também avisa se o PR mexe em arquivo de fora da tarefa. Depois, abra a aba Pull requests do repositório e confira se o SEU pull request está lá e se o robô respondeu: se ele não aparece, a entrega não chegou (o fork sozinho não entrega nada).

Checklist antes de abrir o PR (a Definição de Pronto aplicada à Sprint 1):

- Segui o formato: condição/ação/exceção, “O sistema deve...”, número no RNF.
- Todo item tem ID e origem; nenhum ID repetido.
- Uma coisa por linha.
- Só mexi no MEU arquivo.
- Nenhum dado pessoal.

7.1 Se o pull request voltar com pedido de mudança

Pedido de mudança não é bronca: é o review fazendo o trabalho dele. Leia o comentário do professor ou do Tech Lead, abra o MESMO arquivo no SEU fork, na MESMA branch do pull request (o nome aparece no topo do PR), faça o ajuste e faça um novo commit. O pull request se atualiza sozinho; não abra outro. Depois, responda o comentário dizendo o que mudou.

7.2 Se você faltou na Sprint 1

A sua tarefa não fica travada: se ela ainda estiver Livre, pegue com /eu-pego; se já estiver com alguém, escolha outra Livre do seu squad ou fale com o professor. Para a entrevista, use as anotações dos colegas do squad (peça num comentário da Issue) e o entrevista.md, assim que receber o merge. O prazo é o início da Sprint 2 (05/11), e o passo a passo desta seção funciona de casa, só com o navegador.

8. Sprint Review e retrospectiva

Na review, cada squad tem 3 minutos para mostrar na tela o pull request de cada dev. O PO (o professor) confere o critério de aceite da tarefa: se está tudo certo, aprova e faz o merge, e a tarefa vai para Pronto; se falta algo, comenta no PR, e o dev ajusta na mesma branch. Quem não terminou entrega de casa até o início da Sprint 2 (05/11), avisando o squad na Issue.

Na retrospectiva (5 minutos), o time olha para o jeito de trabalhar, e não para o conteúdo: o que funcionou, o que atrapalhou e qual UMA coisa vamos mudar na Sprint 2.

9. Erros comuns na descoberta

- Escrever o que o time ACHA, e não o que a cliente disse.
- Esquecer de perguntar pela exceção: a regra fica pela metade.
- RNF sem número (“rápido”, “seguro”): não dá para testar.
- RF com duas coisas na mesma linha, ou com coisa de outro squad.
- Item sem origem: ninguém sabe de onde veio.
- Proposta de valor genérica no Canvas (“qualidade”).

10. Antes da Sprint 2 (05/11)

- PR da Sprint 1 aberto, ajustado e aceito.
- Ler os requisitos dos colegas do seu squad: vocês vão precisar deles.
- Rever a Aula 4 (casos de uso) e a história de usuário (Aula 13).
- A Sprint 2 transforma estes requisitos em histórias de usuário e casos de uso.

Glossário

Termo	Significado
Business Model Canvas	O modelo de negócio em uma página, com 9 blocos.
Proposta de valor	Por que o cliente escolhe o negócio, e não o concorrente.
Descoberta	A fase de entender o problema e o negócio antes de construir.
Entrevista	Técnica de levantamento com perguntas abertas, números e exceções.

RN / RF / RNF	Regra de negócio / requisito funcional / requisito não funcional.
Origem	De onde veio o item: uma pergunta da entrevista ou uma regra.
MoSCoW	Prioridade: Must, Should, Could, Won't.
/eu-pego · /largar	Comandos, nos comentários da Issue, para pegar e devolver uma tarefa.
Sprint Review	O momento de mostrar o que ficou pronto e o PO aceitar ou devolver.

Perguntas de revisão

1. Para que serve o Business Model Canvas num projeto de software?
2. Qual a diferença entre Canais e Relacionamento no Canvas? Dê um exemplo do TechBurger para cada.
3. Por que a Estrutura de custos é o último bloco a ser preenchido?
4. Na entrevista, por que perguntar pela exceção ("e quando não dá?")?
5. Transforme a fala "a gente só entrega até 5 km; mais longe, o cliente busca" em regra, RF e RNF.
6. Por que cada dev da Sprint 1 cuida de um arquivo diferente?

Quiz relâmpago para casa

Respostas no Gabarito.

1. Em que bloco do Canvas entra "gateway de pagamento"?
2. "O app deve ser bonito" é um bom RNF? Por quê?
3. Que comando, num comentário da Issue, faz o robô devolver a tarefa?
4. O que significa a sigla RN-PAG-02?
5. Se a sua tarefa não ficou pronta na aula, até quando você pode entregar?